

SCS-RG

Multi-Hazard Smart Campus Safety & Response Grid

A real-time campus emergency monitoring and response platform. Independent sensor zones stream **raw** readings; the backend is the sole authority for validation, risk fusion, state classification, incident lifecycle, response ranking and actuation. A React command dashboard renders the live picture over REST snapshots and Socket.IO events.

The rule everything else follows: **a sensor node is never trusted with a computed value**. A payload carrying its own `riskScore` or `state` is rejected with `400`, not quietly ignored.

1 System Architecture — docs/architecture.md

2 Circuit Diagram — docs/circuit-diagram.md

3 API Documentation — docs/api.md

4 Database Schema — docs/database-schema.md

5 Risk Fusion Formula — docs/risk-fusion.md

Generated 2026-07-26 from `docs/*.md` by `pnpm docs:pdf` — the markdown sources in the repository are authoritative. Endpoint-level detail is also served as OpenAPI at `/api/v1/openapi.json` (`docs/openapi.json`) and interactively at `/api/v1/docs`.

System Architecture

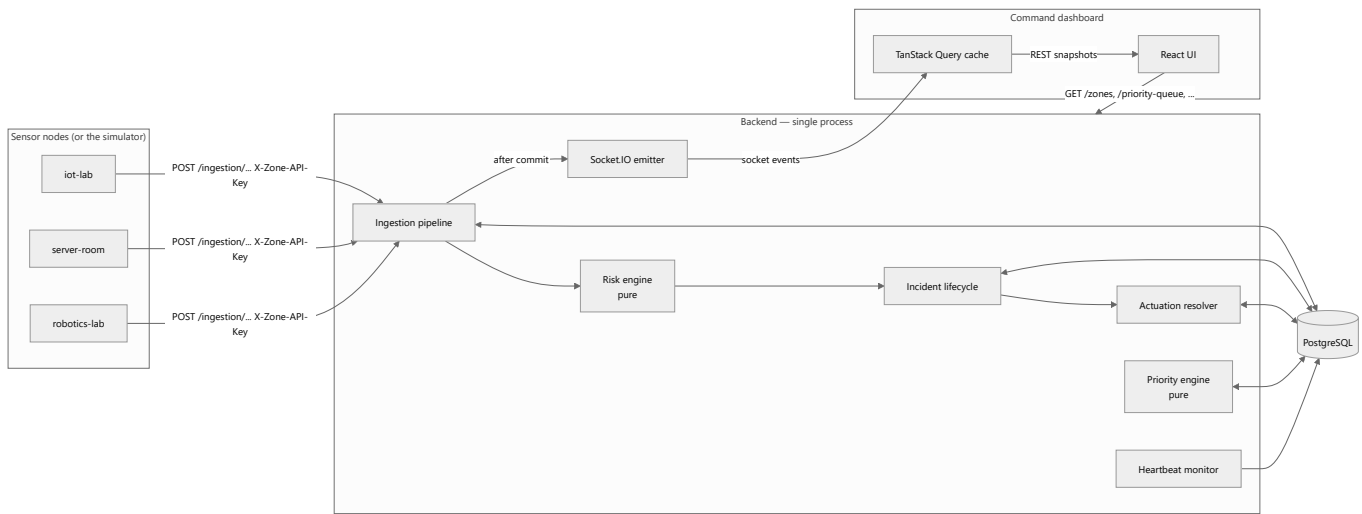
docs/architecture.md

The one rule everything else follows

A sensor node submits **raw readings only**. It never submits a risk score, a state, a priority or an incident status — and if it tries, the request is rejected with `400` rather than having the field quietly ignored. Every computed value in the system is produced by the backend, from data the backend validated itself.

That single rule is what makes the rest of the design fall out: if the node is never trusted, the backend must own validation, debouncing, fusion, classification, the incident lifecycle, ranking, actuation and the audit trail.

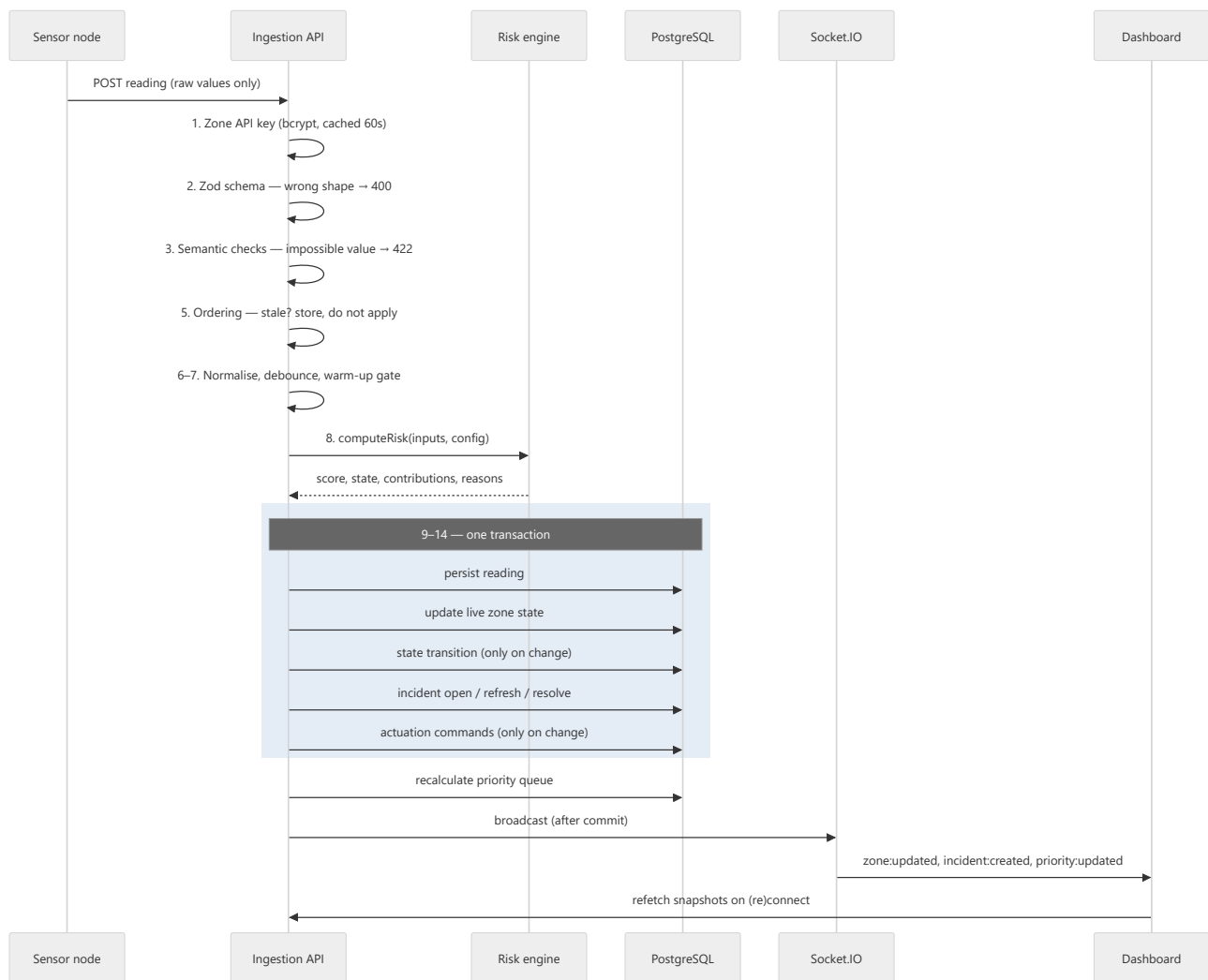
Shape



Packages

Package	Responsibility
<code>packages/shared</code> (<code>@scsrg/shared</code>)	The wire contract: domain enums, Zod schemas, <code>ApiResponse<T></code> , the Socket.IO event map. Both apps typecheck against it, so a DTO cannot drift on one side only.
<code>backend</code>	Express 5 + Prisma + Socket.IO. The sole authority for every computed value.
<code>frontend</code>	React 19 + Vite + Tailwind 4 + shadcn/ui. A view over the backend; it computes no hazard state of its own.

Data flow for one reading



Steps 9–14 are one transaction on purpose. A crash between "reading stored" and "transition written" would leave the database asserting two different things about the same moment; wrapping them means the alternative is that neither happened. The broadcast sits deliberately *outside* the transaction, so a rolled-back write never announces itself and no socket write holds a row lock.

Backend layering

```
routes → controller → service → repository → Prisma
      ↘ pure engine (no I/O)
```

- **Controllers** parse, delegate and shape. No business rule lives in one.
- **Services** hold the rules and never import the Prisma client.
- **Repositories** are the only place Prisma appears; each accepts either the root client or an open transaction, so a service can compose several inside one atomic unit.
- **Engines** (`risk-engine`, `priority-engine`, `actuation.resolver`) are pure functions. No clock, no I/O, no database — time and configuration are passed in. That is what makes their tests deterministic and free of `sleep()`.

In-memory state

Debounce counters, the gas warm-up window, occupancy debounce, water phase and recovery counters live in per-zone maps. Every one of them is a **cache**, never the only copy: each exposes `rehydrate()`, and the bootstrap sequence rebuilds all of them from stored readings before the HTTP listener binds. Killing the process loses nothing.

Frontend state

TanStack Query owns all server data. Socket events **patch or invalidate** that cache — they never write to a parallel store the UI reads instead. On connect and every reconnect the dashboard refetches four snapshots (`/dashboard/summary`, `/zones`, `/incidents?active=true`, `/priority-queue`), so a client that missed events while disconnected converges on the truth rather than showing a confidently stale picture.

De-duplication happens once, at a single socket reader, before events fan out to subscribers. Doing it per subscriber looks equivalent and is not: the first hook to register would consume the `eventId` and every other hook would silently miss the event. The stacked-alert test in `command-center.test.tsx` exists because that is exactly what happened.

What this deliberately is not

- **No queue, cache or second process.** One backend process owns the HTTP server, the socket server, the heartbeat sweeper and the simulator engine. At multi-instance scale the in-memory caches and the heartbeat sweeper would need to move behind a shared store and a single leader.
- **No deployment automation.** Compose for local Postgres and a backend Dockerfile; production infrastructure is documented, not provisioned.
- **No hardware integration.** Actuation is a logged command model with a simulated actuator display.

SECTION 2

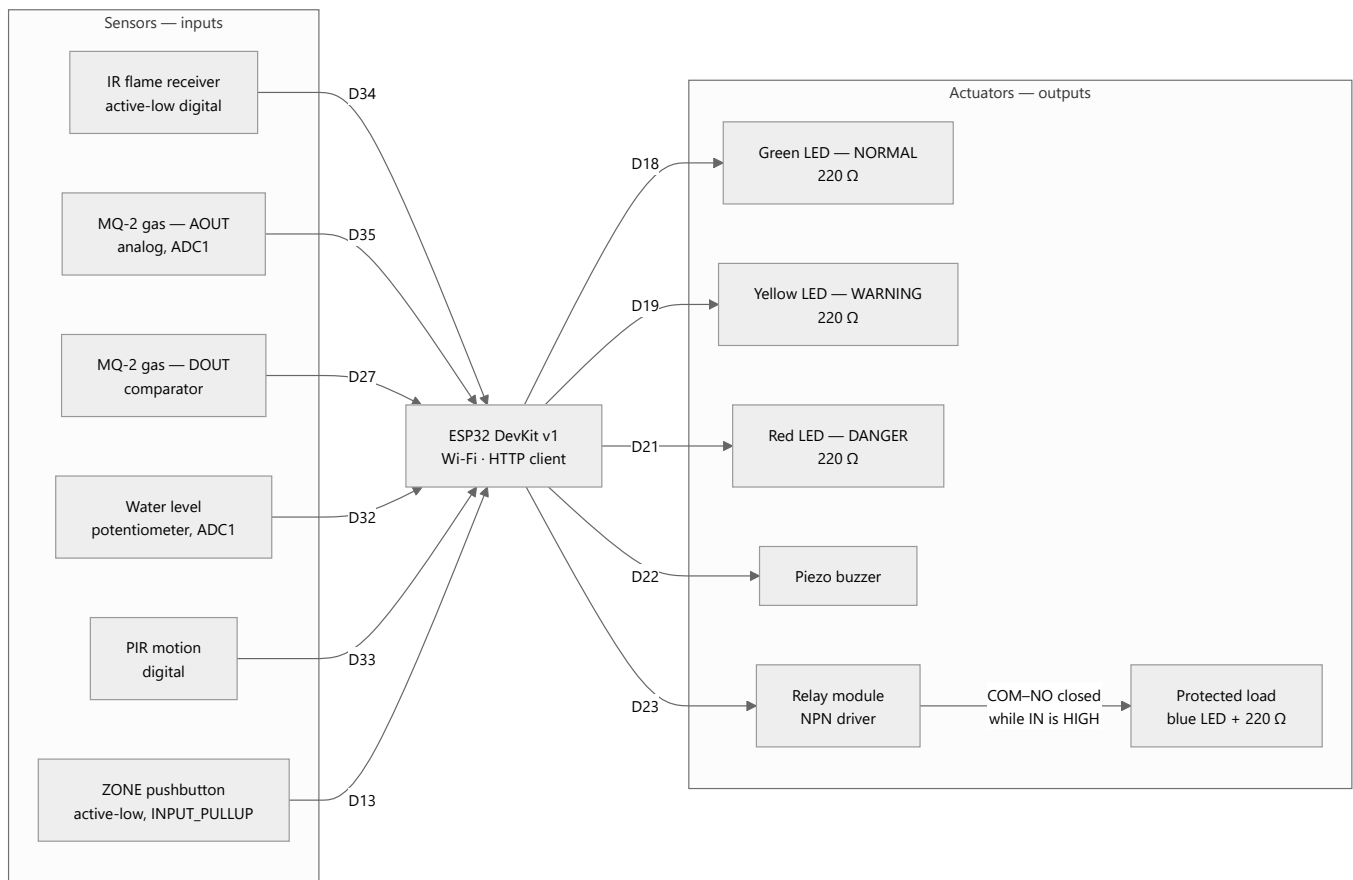
Circuit Diagram

docs/circuit-diagram.md

The hardware side of one zone node: an ESP32-WROOM-32 DevKit v1 sampling flame, gas, water and occupancy, and driving three status LEDs, a buzzer and a power cutoff relay. The board is the same whether it runs on real silicon or in Wokwi — the simulated node speaks the identical HTTP API.

Machine-readable source of truth: `firmware/diagram.json` (the active board) and `firmware/zones/<code>/diagram.json` (one per seeded zone). Pin constants are `PIN_*` in `firmware/include/app_config.h` — that header and the diagrams are the two places a wiring change has to land together.

Schematic



Pin map

Function	GPIO	Direction	Rail	Notes
IR flame — <code>DAT</code>	34	in	3V3	Active-low; bursts latched for <code>IR_FLAME_HOLD_MS</code>
MQ-2 gas — <code>AOUT</code>	35	in (analog)	VIN	ADC1; linearised to ppm, then normalised to 0..1 on the wire
MQ-2 gas — <code>DOUT</code>	27	in	VIN	Comparator line; biased — see <code>GAS_DIGITAL_ACTIVE_LOW</code>
Water level — <code>SIG</code>	32	in (analog)	3V3	Potentiometer stands in for the float/probe in simulation
PIR motion — <code>OUT</code>	33	in	3V3	Occupancy
ZONE select button	13	in	—	To GND, <code>INPUT_PULLUP</code> ; press = next room on the multi board
Status LEDs G / Y / R	18/19/21	out	—	220 Ω series resistor each, cathodes to the GND rail
Buzzer	22	out	—	<code>tone()</code> on an LEDC channel

Function	GPIO	Direction	Rail	Notes
Relay — <code>IN</code>	23	out	VIN	HIGH = powered, LOW = cut
Serial console	TX0/RX0	—	—	115200 baud; every rejection body is printed verbatim

Every analog channel sits on **ADC1**. ADC2 shares hardware with the Wi-Fi radio, so an ADC2 read returns garbage the moment the radio is up — which for this node is always.

The flame receiver, PIR and potentiometer run from **3V3**; the MQ-2 and the relay module take **VIN** (5 V), because neither works reliably at 3.3 V. All grounds are common: sensor grounds to `GND.2`, the output-side breadboard rail to `GND.1`.

The relay is wired to fail safe



The load hangs off the **NO** terminal, not NC. The relay closes COM–NO while `IN` is HIGH, and HIGH is what the firmware drives when the backend has *not* ordered a cutoff — so the load is energised exactly when it should be. A dead, unflashed or unpowered board leaves `IN` low, COM falls back to NC, and the load stays de-energised.

Read the command naming carefully, because it is inverted against the pin: `ACTIVATE_RELAY` carries `relayCutoff: true` and means *cut the power*, so it drives `PIN_RELAY LOW`. `DEACTIVATE_RELAY` restores it. The firmware inverts this explicitly and says so at the call site; `ActuatorState` in `actuation.resolver.ts` is the authority on which state wants which.

Per-zone boards

One firmware, three boards. A zone carries only the sensors it has a row for in the database — reporting a channel the zone lacks is a `422 SENSOR_NOT_CONFIGURED`, so the wiring and the seed data have to agree.

Zone	Populated channels	Omitted
<code>iot-lab</code>	flame (34) · gas (35, 27) · PIR (33)	water
<code>robotics-lab</code>	flame (34) · gas (35, 27) · PIR (33)	water
<code>server-room</code>	flame (34) · water (32) · PIR (33)	gas
<code>multi</code>	all of the above + ZONE button (13)	—

`pnpm firmware:config -- server-room --write` rewrites both `include/zone_secrets.h` and the active `diagram.json`, so credentials and board layout cannot drift apart. The outputs — three LEDs, buzzer, relay — are identical on every variant.

What the board is allowed to decide

Nothing that reaches the wire. The node converts device units to the normalised 0..1 contract on-device (the calibration constants live there), and sends raw values only:

Channel	Device units	On the wire	Conversion constant
Gas	200–10000 ppm	<code>0..1</code>	<code>GAS_REPORT_FULL_SCALE_PPM</code>
Water	0–100 %	<code>0..1</code>	—
Flame	active-low digital	boolean	<code>IR_FLAME_HOLD_MS</code> latch
PIR	digital, or stale	boolean / <code>null</code>	<code>SENSOR_TIMEOUT_MS</code>

Fire debounce and the gas warm-up window are **backend** rules: the node reports the raw line every publish interval and lets the backend decide when a flicker becomes a fire. A PIR that has timed out reports `occupancyDetected: null`, never `false` — the schema models an unavailable sensor as unknown, and `false` would read as "nobody is in danger here".

The node does compute a local risk score, and it never transmits it. It exists for one case: after `BACKEND_AUTHORITY_GRACE_MS` without a successful call, the node stops obeying stale commands and its own state machine takes the pins back, so a node that loses the network keeps alarming locally instead of going dark. Keep that grace period above `ZONE_OFFLINE_TIMEOUT_MS` so the backend has already marked the zone `OFFLINE` before the node stops listening.

Bringing the board up

```
pnpm db:up && pnpm db:seed && pnpm dev    # backend must exist before the node does
pnpm firmware:config -- iot-lab --write  # writes zone UUID + API key + diagram
cd firmware && pio run                    # then start Wokwi, or: pio run -t upload -t monitor
```

Reaching a backend on your own machine from Wokwi needs the **Private IoT Gateway** — that is what makes `host.wokwi.internal` resolve. Full build and run instructions, including headless Wokwi CI, are in `firmware/README.md`.

SECTION 3

API Documentation

docs/api.md

Base path `/api/v1`. Interactive documentation with live examples: `http://localhost:4000/api/v1/docs`. The machine-readable document is at `/api/v1/openapi.json` and can be emitted with `pnpm docs:openapi`.

Envelope

Every response, without exception:

```
{ "success": true, "data": {}, "meta": {} }
```

```
{
  "success": false,
  "error": {
    "code": "VALUE_OUT_OF_RANGE",
    "message": "Gas level must be between 0 and 1 - received 1.5.",
    "details": [
      { "path": "sensors.gasLevel", "message": "Value outside the 0-1 range" }
    ]
  }
}
```

`meta` carries pagination on list endpoints: `page`, `pageSize`, `total`, `totalPages`, `hasNextPage`.

Status codes

Code	Meaning here
200	OK
201	Created — including an accepted sensor reading
400	Malformed input. Wrong shape, wrong types, or a forbidden field
401	Unauthenticated, or an invalid/revoked zone key
403	Authenticated but not permitted, or a deactivated zone
404	Not found
409	Conflict — duplicate reading, already acknowledged
413	Body over the 1 MB limit
422	Valid shape, impossible values
429	Rate limited
500	Unexpected

The `400` / `422` distinction is load-bearing. `{"gasLevel": "high"}` is the wrong *shape* — 400. `{"gasLevel": 1.5}` is the right shape carrying an impossible *value* — 422. They mean different things to whoever is debugging a node.

Error codes

`VALIDATION_ERROR` · `UNAUTHENTICATED` · `INVALID_CREDENTIALS` · `FORBIDDEN` · `NOT_FOUND` · `DUPLICATE_READING` · `ALREADY_ACKNOWLEDGED` · `VALUE_OUT_OF_RANGE` · `SENSOR_NOT_CONFIGURED` · `INVALID_TIMESTAMP` · `ZONE_INACTIVE` · `INVALID_ZONE_KEY` · `RATE_LIMITED` · `CONFLICT` · `PAYLOAD_TOO_LARGE` · `INTERNAL_ERROR`

Endpoints

Auth

Method & path	Auth	Notes
POST /auth/login	public	Rate-limited 5/min/IP. Returns token + user.
GET /auth/me	JWT	Current user and role.

Zones

Method & path	Auth	Notes
GET /zones	JWT	All zone statuses in one request.
GET /zones/:zoneId	JWT	Detail incl. configuration. Accepts an id <i>or</i> a code (<code>iot-lab</code>).
GET /zones/:zoneId/readings	ADMIN	Paginated, date-filterable raw history.
GET /zones/:zoneId/timeline	JWT	Transitions and incidents merged.
GET /zones/:zoneId/transitions	JWT	State-change history.
GET /zones/:zoneId/system-health	ADMIN	Per-zone connectivity.

Ingestion — zone API key only

Method & path	Notes
POST /ingestion/zones/:zoneId/readings	The full pipeline. Returns the backend's own verdict.
POST /ingestion/zones/:zoneId/heartbeat	Liveness without a reading.
GET /ingestion/zones/:zoneId/commands	Pending actuation commands.
POST /ingestion/zones/:zoneId/commands/:commandId/complete	Confirm execution.

Request — raw values only:

```
{
  "readingId": "iot-lab-1042",
  "sequenceNumber": 1042,
  "capturedAt": "2026-07-25T10:30:15.000Z",
  "sensors": {
    "fireDetected": true,
    "gasLevel": 0.72,
    "waterLevel": 0,
    "occupancyDetected": true
  }
}
```

A payload containing `riskScore`, `state`, `priority` or `incidentStatus` is **rejected with 400**, not silently stripped. `occupancyDetected: null` means the sensor is unavailable — never send `false` to mean "unknown".

Response — the backend's verdict, returned so a node or the simulator can display what the system actually decided:

```

{
  "success": true,
  "data": {
    "accepted": true,
    "readingId": "iot-lab-1042",
    "validationStatus": "ACCEPTED",
    "appliedToLiveState": true,
    "computation": {
      "riskScore": 72.5,
      "state": "CRITICAL",
      "contributions": { "fire": 40, "gas": 17.5, "water": 0, "occupancy": 15 },
      "reasons": [
        "Sustained flame confirmed after debounce (5 consecutive readings) (+40)",
        "Gas level is 70% of configured range (+17.5)",
        "Zone is currently occupied (+15)",
        "Combined score crosses the CRITICAL threshold (65)"
      ]
    },
    "zoneState": "CRITICAL",
    "incidentId": "8c1...",
    "actuationCommandIds": ["a1...", "b2...", "c3..."]
  }
}

```

A stale reading returns `201` with `validationStatus: "ACCEPTED_OUT_OF_ORDER"` and `appliedToLiveState: false` — stored for audit, barred from moving live state, creating a transition, opening an incident or issuing actuation.

Incidents

Method & path	Auth	Notes
GET <code>/incidents</code>	JWT	Filters: <code>from</code> , <code>to</code> , <code>zoneId</code> , <code>status</code> , <code>active</code> , <code>hazardType</code> , <code>acknowledgedBy</code> ; paginated.
GET <code>/incidents/:incidentId</code>	JWT	Detail with timeline, surrounding readings and actuation. <code>404</code> on unknown id.
GET <code>/incidents/:incidentId/timeline</code>	JWT	Ordered events.
POST <code>/incidents/:incidentId/acknowledge</code>	JWT	<code>200</code> / <code>409</code> . Optional <code>note</code> .

Priority and dashboard

Method & path	Auth	Notes
GET <code>/priority-queue</code>	JWT	Ranked active incidents with breakdown and reasons . Empty is <code>[]</code> , never <code>null</code> .
GET <code>/dashboard/summary</code>	JWT	Everything the top summary bar needs, in one request.

Admin — all `ADMIN`

Method & path	Notes
POST <code>/admin/zones</code>	Returns the plaintext API key once .
PATCH <code>/admin/zones/:zoneId</code>	Update; <code>isActive: false</code> is the only removal path.
POST <code>/admin/zones/:zoneId/credentials</code>	Rotate the API key; revokes the previous one.
POST <code>/admin/zones/:zoneId/overrides</code>	Requires a <code>reason</code> of ≥ 5 characters.

Method & path	Notes
PATCH /admin/sensors/:sensorId	Sensor configuration and status.
GET /admin/system-health	Full health picture.
GET /admin/audit-logs	Paginated, filterable.
GET /admin/users · PATCH /admin/users/:userId/role	Role management; the last admin cannot demote themselves.

Simulator — all ADMIN

GET /simulator/status · POST /simulator/zones/:zoneId/start · /stop · PATCH /simulator/zones/:zoneId/state · POST /simulator/zones/:zoneId/fault · POST /simulator/scenarios/:scenarioId/run · POST /simulator/stop-all

Reports (bonus 3)

Method & path	Auth	Notes
POST /reports/natural-language	JWT	Extraction via the configured AI chain, falling back to the deterministic extractor. Result is PENDING and influences nothing.
GET /reports	JWT	List, filterable by status.
POST /reports/:reportId/confirm	ADMIN	Only now can it contribute a bounded priority bonus. 409 if the report was already ruled on.
POST /reports/:reportId/reject	ADMIN	Files the report as rejected; it keeps influencing nothing. 409 if the report was already ruled on.

Advisory (bonuses 1 and 2)

Method & path	Auth	Notes
GET /trend/:zoneId	JWT	Moving average and slope. Never affects state.
GET /prediction/:zoneId	JWT	P(CRITICAL within 60s). Cannot actuate.

Health

GET /health (liveness) · GET /health/ready (readiness incl. database). Both are outside the versioned prefix and outside rate limits, so an orchestrator probe can never be throttled.

Real-time events

Socket.IO at /socket.io, JWT-authenticated in the handshake (auth.token). An unauthenticated handshake is refused.

Rooms: dashboard (all authenticated clients), zone:<zoneId> (detail views), admin (admin-only payloads such as simulator traffic).

Server → client: zone:updated · zone:state-changed · reading:accepted · incident:created · incident:updated · incident:acknowledged · incident:resolved · priority:updated · sensor:offline · system:health · actuation:command · simulator:payload · simulator:response · simulator:status · trend:updated · prediction:updated · report:created

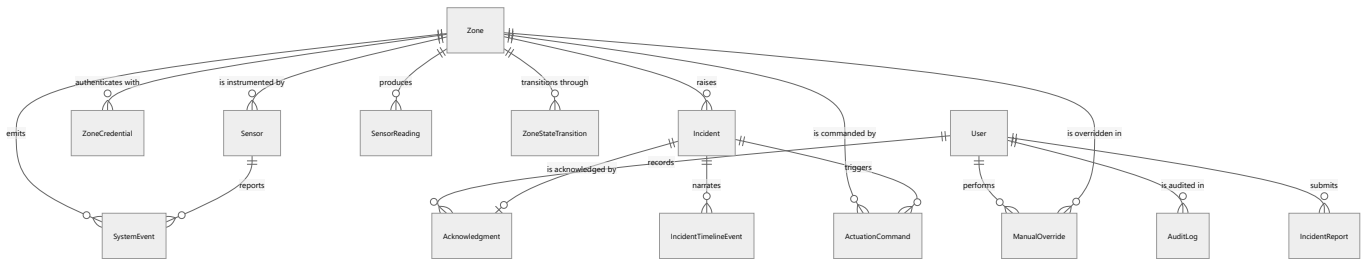
Every payload carries { eventId, emittedAt, ...data }. Sockets are never the only source of truth: on connect and every reconnect the dashboard refetches /dashboard/summary, /zones, /incidents?active=true and /priority-queue.

Database Schema

docs/database-schema.md

Normalised PostgreSQL 18, managed with Prisma migrations. Full definition: `backend/prisma/schema.prisma`.

Entity relationships



Tables

Model	Purpose	Load-bearing constraints
User	Dashboard accounts	UNIQUE(email); passwordHash bcrypt cost 12
Zone	Monitored area and its live projection	UNIQUE(code); assetImportance 0–8; soft delete via isActive
ZoneCredential	Per-zone API key	apiKeyHash bcrypt; nullable revokedAt; index (zoneId, revokedAt)
Sensor	Per-zone sensor configuration	UNIQUE(zoneId, type); isCritical; free-form configuration JSON
SensorReading	Immutable raw + computed record	UNIQUE(readingId), UNIQUE(zoneId, sequenceNumber), FK → Zone onDelete: Restrict
ZoneStateTransition	State-change audit	index (zoneId, createdAt)
Incident	Hazard event	partial UNIQUE(zoneId) WHERE status IN ('OPEN', 'ACKNOWLEDGED'); FK → Zone onDelete: Restrict
Acknowledgment	Who acknowledged, when	UNIQUE(incidentId)
IncidentTimelineEvent	Ordered narrative	index (incidentId, createdAt)
ActuationCommand	LED / buzzer / relay command log	indexes (zoneId, requestedAt), (status)
ManualOverride	Admin action record	FKs → Zone, User
AuditLog	Security-relevant actions	indexes (userId, createdAt), (entityType, entityId), (action)
SystemEvent	Health / validation / offline events	indexes (createdAt), (type, severity)
IncidentReport	Natural-language field report (bonus 3)	status PENDING / CONFIRMED / REJECTED
ReadingHourlyAggregate	Retention rollup	UNIQUE(zoneId, hour)

The two constraints Prisma's DSL cannot express

Both live in a hand-written migration, `20260724192500_partial_unique_indexes`.

One active incident per zone

```
CREATE UNIQUE INDEX incident_one_active_per_zone
ON "Incident" ("zoneId")
WHERE "status" IN ('OPEN', 'ACKNOWLEDGED');
```

This is the no-duplicate-incident guarantee. A score oscillating around the threshold, or two concurrent ingestion requests for the same zone, cannot create a second active incident because Postgres refuses the insert. The application catches the violation and treats it as "someone else already opened it" — it never re-implements the rule.

`schema-constraints.test.ts` asserts the second insert fails, and that a new incident is allowed once the previous one is `RESOLVED`.

Partial index for the active set

```
CREATE INDEX incident_active_started_at
ON "Incident" ("startedAt" DESC)
WHERE "status" IN ('OPEN', 'ACKNOWLEDGED');
```

The priority queue and dashboard summary read the active set constantly. A partial index over three rows beats a full index over 270.

Referential integrity

`Incident.zoneId` and `SensorReading.zoneId` use `onDelete: Restrict`. A zone with history **cannot** be deleted — `PATCH /admin/zones/:zoneId` with `isActive: false` is the supported removal path, and the integration test asserts the hard delete fails.

The `isDuplicate` column

Two different things are easy to confuse here:

- An **exact duplicate** (same `readingId`, or same `(zoneId, sequenceNumber)`) is rejected at the unique constraint with `409` and logged as a `SystemEvent`. It never creates a second row, so it can never be counted twice.
- `isDuplicate = true` marks an **accepted** reading whose sensor payload was byte-identical to the immediately preceding accepted reading. It is a real signal — "nothing changed" — used to suppress redundant timeline noise.

Performance gate

With 24 838 readings and 272 incidents seeded, the hot query — *all CRITICAL or active incidents from the last 24 hours across all zones* — must complete in under 50 ms using an index.

```
$ pnpm db:explain

Index used:      incident_active_started_at
Execution time: 0.39 ms (budget 50 ms)

✓ Performance gate passed.
```

`scripts/explain-hot-queries.ts` runs `EXPLAIN (ANALYZE, BUFFERS)` and **exits non-zero** on a sequential scan or a runtime over budget. Dropping the index breaks the build; a gate that only prints is not a gate.

Seed data

`pnpm db:seed` is idempotent and produces:

- Two development accounts (see the README — they are development-only).
- Three zones with their sensors, asset importance and one API credential each.
- $\geq 10\,000$ historical readings spread over seven days.
- Resolved incidents with complete timelines and actuation records.
- Exactly one acknowledged incident.
- Sample audit logs and system events.

Zone API keys are printed once and written to `backend/.dev-zone-keys.json` and `backend/.env.simulator`, both gitignored. Re-running the seed rotates them — the previous plaintext is unrecoverable, so leaving the old hash in place would mean an unusable credential.

Risk Fusion Formula

docs/risk-fusion.md

The formula

```
riskScore = 40·fireSignal
           + 25·normalizedGasLevel
           + 20·normalizedWaterLevel
           + 15·occupancyFactor
```

Clamped to `[0, 100]`, rounded to two decimals.

Classification	Score
<code>SAFE</code>	0 – 29.99
<code>WARNING</code>	30 – 64.99
<code>CRITICAL</code>	65 – 100

Weights and thresholds live in `backend/src/config/risk.config.ts` and are overridable by environment variable. The engine itself (`risk.service.ts`) contains no literal numbers.

`OFFLINE` is deliberately absent from the classifier. It is a connectivity fact, not a score, and is applied afterwards by the zone-state service. No score can make a zone `OFFLINE`, and `OFFLINE` is never inferred as `SAFE`.

Why these weights

The weights are chosen so the arithmetic encodes a response policy rather than a vague sense of severity.

- **Fire = 40.** The only signal that alone reaches `WARNING`, and the only one that combined with a single other saturated hazard reaches `CRITICAL`. That matches a confirmed-flame policy: flame plus anything means cut the power.
- **Gas = 25.** Gas at 100 % plus a full room (40) stays `WARNING`. A saturated gas reading without flame is an evacuate-and-ventilate event, not a cut-the-power event. Gas 100 % *with* confirmed flame is exactly 65 — `CRITICAL` at the boundary, by construction.
- **Water = 20.** Below gas because water damages equipment on a slower clock than fumes injure people. Full water plus flame is 60 — `WARNING` — and only crosses when a third signal joins it.
- **Occupancy = 15.** People are a severity multiplier, never a hazard. Occupancy alone is 15, comfortably inside `SAFE`. A zone cannot become dangerous merely by being busy.

Worked example from the specification:

```
{
  "riskScore": 72.5,
  "state": "CRITICAL",
  "contributions": { "fire": 40, "gas": 17.5, "water": 0, "occupancy": 15 },
  "reasons": [
    "Sustained flame confirmed after debounce (5 consecutive readings) (+40)",
    "Gas level is 70% of configured range (+17.5)",
    "Zone is currently occupied (+15)",
    "Combined score crosses the CRITICAL threshold (65)"
  ]
}
```

Reasons are produced by a pure `explain()` function, one rule per contributing signal, so the dashboard never has to reconstruct *why* a zone is in its state.

Sensor processing rules

These run **before** fusion. The engine receives signals that have already been debounced and gated; it decides how much they weigh, not whether they count.

Fire debounce — asymmetric on purpose

`fireSignal` becomes 1 only after `FIRE_DEBOUNCE_CONSECUTIVE` (default 5) consecutive positive readings. It returns to 0 only after `FIRE_CLEAR_CONSECUTIVE` (default 5) consecutive negatives.

The asymmetry is the point. Confirming slowly stops a flicker raising an alarm; clearing slowly stops a momentary sensor dropout *during a real fire* silencing one. The cost of a missed fire is not the cost of a spurious one, so the two directions are not treated alike.

A four-reading flicker contributes 0 and says so in the reasons:

Flame seen on 4 reading(s) — below the debounce threshold, contributing 0

Gas warm-up

A metal-oxide gas sensor reads high for its first seconds of life. During `GAS_WARMUP_MS` (5 000 in demo, 30 000 in production) the gas contribution is forced to 0, the sensor reports `WARMING_UP`, and gas can raise neither state nor an incident. The suppression is always stated:

Gas sensor is warming up (4s remaining) — reading suppressed, contributing 0

The window restarts on reconnection, because a node that just rebooted has a cold sensor again.

Water phase

Derived from the same normalised value the engine weighs; it changes presentation and hazard classification, never the arithmetic.

Phase	Level
<code>DRY</code>	< 0.15
<code>RISING</code>	0.15 – 0.59
<code>CRITICAL</code>	≥ 0.6
<code>RESET</code>	falls below 0.1 <i>after</i> having risen

A permanently dry probe reads `DRY`, not `RESET` — `RESET` means "it receded", which is different from "it was never wet".

Occupancy — the subtle one

A disconnected occupancy sensor yields `UNAVAILABLE`, **never** `occupancy: false`. "Nobody is here" and "we don't know" are different facts, and conflating them is how a control room decides not to send anyone.

Unknown occupancy therefore has two different answers depending on the question:

Question	Answer	Why
How much risk does it add?	0	The system does not fabricate hazard from missing data.
Should we treat the room as occupied when dispatching?	Yes	Dispatch fails safe.

Both halves appear in the reading's reasons:

Occupancy sensor unavailable — not counted toward risk, but treated as occupied for response priority

Confirmed occupancy changes are debounced over `OCCUPANCY_DEBOUNCE_READINGS` (default 3) so a passer-by cannot spam events.

Recovery hysteresis

Leaving `CRITICAL` requires the score to sit below `RISK_THRESHOLD_CRITICAL - STATE_HYSTERESIS` ($65 - 5 = 60$) for `RECOVERY_CONSECUTIVE_READINGS` (default 3) consecutive accepted readings.

Without it, a score oscillating around 65 would open and close incidents and toggle the relay several times a second — at exactly the moment an operator most needs a stable picture. The integration test drives $63 \leftrightarrow 66$ across twenty readings and asserts **one** incident row.

Test coverage

`risk-engine` and `priority-engine` are gated at $\geq 90\%$ lines and branches. The boundaries are asserted exactly:

Input	Score	State
occupancy + water 0.7495	29.99	<code>SAFE</code>
occupancy + water 0.75	30	<code>WARNING</code>
fire + gas 0.9996	64.99	<code>WARNING</code>
fire + gas 1.0	65	<code>CRITICAL</code>

There is no `sleep()` anywhere in the unit suite. Every timing rule takes an injected clock and the tests advance it explicitly.